

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	P. Lohith Krishna	Reg. No.:	192325122
Date:	19.08.2026	Duration:	60 minutes

Code of Conduct

I P. Lohith Krishna 192325122 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Lohith Krishna

Annexure A – Code Review & Repository Evaluation

Smart Food Delivery and Order Management System

1. PROBLEM OVERVIEW

1.1 Introduction

The Smart Food Delivery and Order Management System is a software application designed to help customers, restaurant staff, and delivery partners report, track, and manage delivery and orders in a systematic manner.

Food ordering and delivery becomes difficult when issues are reported through separate channels such as messages, calls, or verbal communication. Manual handling can cause delays, duplicate requests, and limited visibility into the progress of an issue.

The proposed system provides a centralized platform where customers can browse restaurants, select menu items, place orders, make payments, and track delivery status. Administrators and restaurant staff can manage menus, orders, payments, and delivery assignments.

The application focuses on reducing manual coordination, improving response time, maintaining a clear order history, and providing a simple interface for reporting food delivery issues.

1.2 Problem Statement

Customers and restaurant staff often face difficulties in reporting and following up on food ordering and delivery issues because requests may be communicated through informal channels and there is limited visibility into their status.

The major problems include:

- Manual request handling is time-consuming.
- Incomplete order details can delay delivery work.
- Customers may not know which restaurant, menu option, or delivery option best suits their needs.
- Delivery requests are often reported through separate channels.
- Maintaining request records manually is difficult.
- Users require a simple system for viewing order status and updates.

Therefore, a smart software solution is required to combine order placement, order tracking, delivery assignment, and delivery updates into a single system.

1.3 Proposed Solution

The proposed system allows users to submit orders with relevant order details and tracks each request through predefined workflow states.

The system can provide functionalities such as:

1. User registration and login.
2. Food order placement.
3. Restaurant and menu selection.
4. Request delivery assignment to delivery partners.
5. Request status tracking.
6. Service history and resolution tracking.
7. Notification of request updates.
8. Administrator order management.
9. Validation of user inputs.
10. Display of order status and service summaries.

The system is designed using a modular approach so that individual components can be modified or extended without affecting the entire application.

1.4 Project Objectives

The main objectives are:

- To automate food order placement and tracking.
- To reduce delays and errors in manual request handling.
- To provide a simple and user-friendly interface.
- To manage delivery requests and order information.
- To support delivery assignment and planning.
- To provide order status and service summaries.

- To improve maintainability through modular code.
- To maintain project history using Git.
- To provide proper project documentation.
- To validate application functionality through testing.

1.5 Key Functionalities

User Management

Users can register and log in to the application. Authentication helps protect user-specific orders and restaurant and delivery information.

Order Management

Customers can browse restaurants, select menu items, specify delivery details, and add special instructions.

Order Tracking

The system processes submitted orders and assigns workflow states such as Placed, Confirmed, Preparing, Out for Delivery, and Delivered.

Restaurant and Menu Management

Restaurants and menu items can be organized by cuisine, availability, and delivery options so that customers can find suitable choices quickly.

Delivery Planning

The system can help administrators organize orders, delivery assignments, delivery activities, and resolution records.

Order Summary

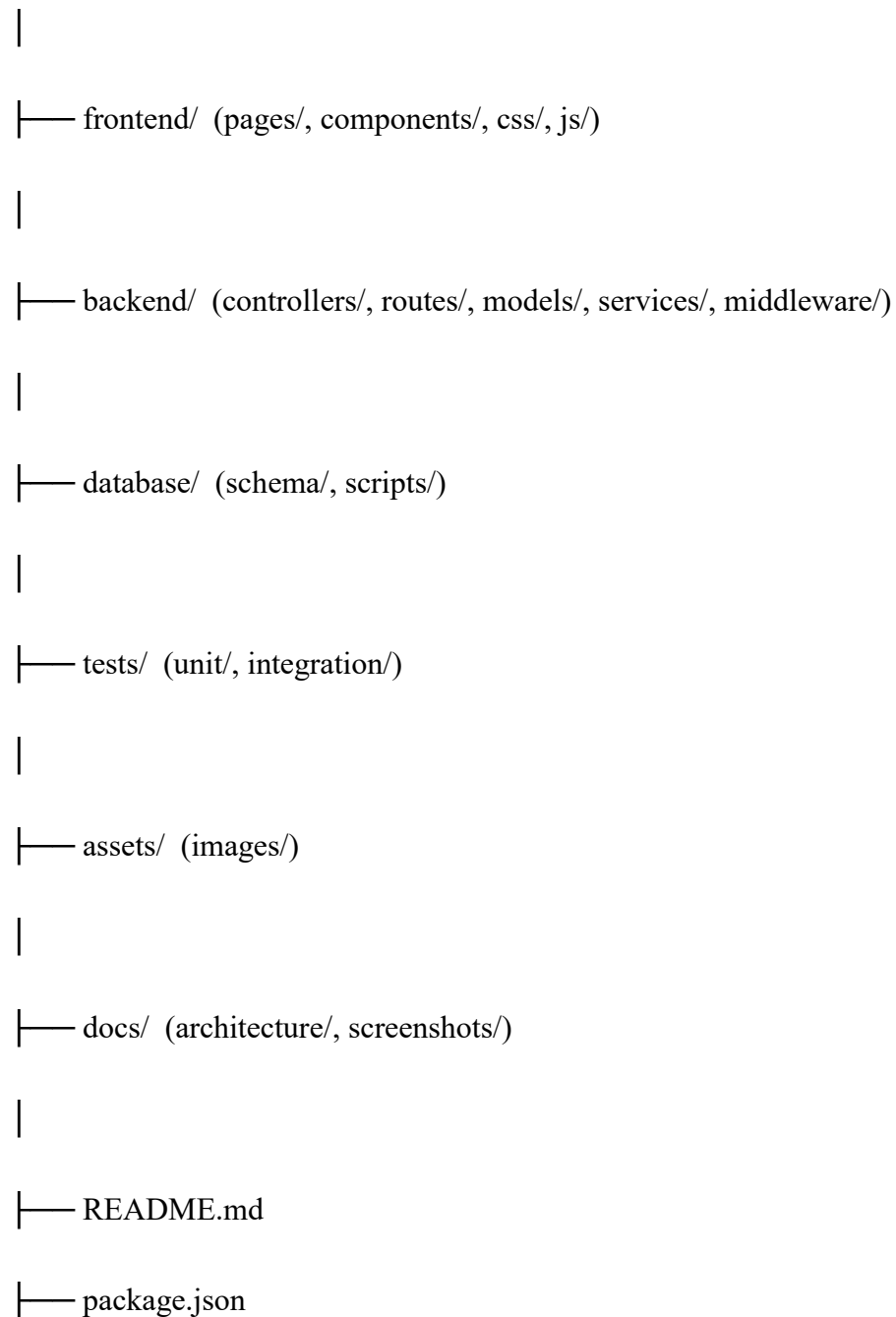
The order status and order history can be presented in a simple summary so that users can understand the progress of their reported issue.

2. REPOSITORY ORGANIZATION

2.1 Repository Structure

The project repository should be organized into logical folders and files. A recommended structure for the system is:

Smart-Food-Delivery-Order-Management/



├── .gitignore

└── LICENSE

Note: If your actual project uses a different structure, replace the above structure with your real GitHub repository structure.

2.2 Folder Organization

The repository is divided according to application responsibilities.

Folder Purpose

frontend User interface and client-side functionality

backend Server-side application logic

controllers Handles order operations

routes Defines application routes

models Represents database entities

services Contains business logic

middleware Handles authentication, validation, and error handling

database Database-related files

tests Testing files

docs Project documentation

assets Images and static resources

This separation improves maintainability because developers can identify the location of specific functionality quickly.

2.3 File Naming Conventions

Meaningful and consistent file names should be used.

Examples: requestController

requestRoutes

userModel

deliveryService

login

register

dashboard

File names should clearly indicate their purpose. Consistent naming reduces confusion during development and debugging.

2.4

Maintainabilit

y

The repository structure supports maintainability by:

- Separating frontend and backend responsibilities.
- Separating business logic from user interface code.
- Keeping database-related files organized.
- Maintaining a separate testing directory.
- Providing project documentation.
- Using meaningful file and folder names.

2.5 Collaboration

A properly organized repository supports collaboration because multiple developers can work on different modules independently.

- For example:
- Developer 1 → User authentication.
- Developer 2 → Service order management.
- Developer 3 → Delivery assignment and tracking.
- Developer 4 → Testing and documentation.
- Screenshot Evidence

Figure 1: GitHub Repository Structure

[Insert Screenshot of your GitHub repository showing folders and files]

The frontend contains the pages and components used by customers, restaurant staff, and administrators.

- The backend contains order processing, authentication, delivery assignment, notification, and reporting logic.
- The database folder contains schema definitions and initialization scripts.
- The tests folder separates unit tests from integration tests.
- The docs folder stores architecture notes, setup information, and screenshot evidence.

The README provides the main project description and execution instructions.

Git should track source code and documentation while excluding generated files and secrets.

These conventions make repository review and collaborative development easier.

3. CODE QUALITY REVIEW

3.1 Code Readability

Readable source code is important for long-term delivery.

The code should use:

- Meaningful variable names.
- Meaningful function names.
- Proper indentation.
- Consistent formatting.
- Appropriate comments.
- Small and focused functions.

For example:

```
function createServiceRequest(title, category) {  
  
  const request = { title, category, status: "Placed" };  
  
  return saveServiceRequest(request);  
  
}
```

This is easier to understand than placing order creation, validation, payment processing, database access, and notification logic inside one large function.

3.2 Modularity

The application should be divided into independent modules.

For example:

User Module

↓

Authentication Module

↓

Order Module



Delivery Assignment Module



Notification Module



Report Module

Each module performs a specific responsibility.

This reduces code duplication and makes future modifications easier.

3.3 Coding Standards

The following coding practices should be followed:

- Consistent indentation.
- Descriptive names.
- Avoid unnecessary duplicate code.
- Use reusable functions.
- Validate user inputs.
- Handle exceptions properly.
- Avoid hard-coded values wherever possible.
- Keep sensitive information outside source code.

3.4 Documentation in Code

Important functions should contain comments or documentation explaining:

- Purpose of the function.
- Input parameters.
- Output.
- Important request-processing logic.

For example:

```
// Creates a new food delivery request after validation.
```

```
function createOrder(restaurantId, items, address) {  
  
    return orderService.create({ restaurantId, items, address });  
  
}
```

3.5 Strengths Identified

1. The major strengths of the implementation can be summarized as:
2. The application solves a practical food delivery problem.
3. The system can centralize restaurant ordering, payment, and delivery tracking.
4. Functional modules can be separated logically.
5. User input can be validated.
6. Git provides project history.
7. The application can be extended with additional food deliveries.

7. Modular design improves maintainability.

3.6 Areas for Improvement

- Possible improvements include:
 - Increase code comments for complex order and payment workflows.
 - Reduce duplicate code.
 - Improve error handling.
 - Use centralized validation.
 - Add automated unit tests.
 - Avoid hard-coded request categories and statuses.
 - Store configuration values separately.
 - Improve security of authentication and payment processing.
 - Add logging for important order operations.
-

4. VERSION CONTROL EVALUATION

4.1 Git Usage

Git is used to maintain the history of source-code changes.

Typical Git workflow:

Working Directory

↓

`git add`

↓

Staging Area

↓

`git commit`

↓

Local Repository

↓

`git push`

↓

GitHub Repository

4.2 Commit History

The commit history should clearly describe changes made to the project.

Good commit messages include:

Add order processing module

Fix order input validation

Add delivery assignment dashboard

Update README installation steps

Fix authentication issue

Add test cases for order placement and tracking

Avoid vague messages such as:

changes

final

test

Meaningful commit messages make it easier to understand the development history.

Screenshot Evidence

Figure 2: Git Commit History

[Insert Screenshot of GitHub commit history]

4.3 Branching Strategy

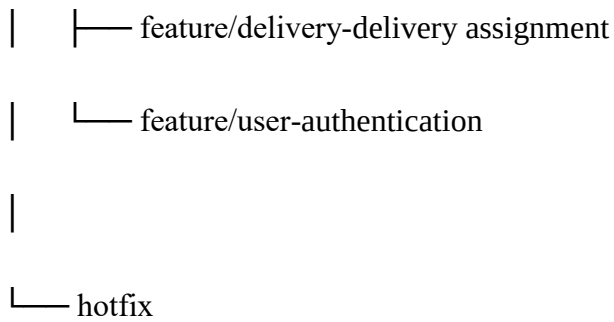
A suitable branching strategy is:

main

|

|— develop

| |— feature/service-request



The main branch should contain stable code.

Feature branches can be used for independent development before merging changes into the main branch.

4.4 Repository Delivery

Repository delivery should include:

- Removing unnecessary files.
- Maintaining a proper .gitignore.
- Updating README.
- Using meaningful commits.
- Reviewing pull requests.
- Keeping dependencies updated.
- Removing unused code.
- Maintaining a stable main branch.

4.5 Importance of Version Control

Git supports collaborative software development by providing:

- Change tracking.
- Version history.
- Branch management.
- Collaboration.
- Backup of source code.
- Ability to revert unwanted changes.
- Easier debugging through commit history.

5. CODE TESTING AND VALIDATION

5.1 Testing Objective

Testing is performed to verify that the application produces the expected results and handles invalid input correctly.

The testing process covers:

- Functional testing.
- Input validation.
- Food order placement testing.
- Authentication testing.
- Request status tracking testing.
- Error handling.

5.2 Test Cases

Test ID	Test Case	Input	Expected Result	Status
TC01	To be tested			
TC02	To be tested			
TC03	To be tested			
TC04	To be tested			
TC05	To be tested			
TC06	To be tested			
TC07	To be tested			
Order Tracking	Existing order ID	Current order displayed	statusTo be tested	

Delivery Update	Order status changed	Customer notified	To be tested
Logout	Logged-in customer	Session terminated	To be tested
TC10 Logout	Logged-in user	Session terminated	To be tested

Replace the Status column with the actual result from your testing.

5.3 Boundary Testing

Boundary values should also be tested.

Examples:

- Empty delivery address.
- Minimum supported cart quantity.
- Maximum supported cart quantity.
- High delivery-fee boundary.
- Missing delivery address.
- Invalid menu category.

5.4 Validation Testing

The system should reject invalid values such as:

Restaurant ID = blank

Restaurant ID = blank

Payment method = invalid value

Email = invalid@email

Appropriate validation messages should be displayed to the user.

5.5 Testing Evidence

Figure 3: User Registration Test

[Insert Screenshot]

Figure 4: Login Test

[Insert Screenshot]

Figure 5: Order Submission Test

[Insert Screenshot]

Figure 6: Order Tracking Dashboard

[Insert Screenshot]

Figure 7: Invalid Input Validation

[Insert Screenshot]

6. REPOSITORY DOCUMENTATION

6.1 README

The repository should contain a README file explaining the project.

A good README should include:

1. Project title.
2. Problem statement.
3. Features.
4. Technologies used.
5. System requirements.
6. Installation instructions.
7. Configuration.

8. Database setup.
9. Application execution.
10. Screenshots.
11. Testing instructions.
12. Future enhancements.

6.2 Installation Guide

A typical installation process is:

Step 1: Clone the repository.

Step 2: Install required dependencies.

Step 3: Configure the database.

Step 4: Configure environment variables.

Step 5: Start the application.

Step 6: Placed the application in the browser.

```
git clone <repository-url>
```

```
cd Smart-Food-Delivery-Order-Management
```

Then install the required dependencies according to the technology used by the project.

6.3 Dependencies

The README should clearly list all dependencies.

Example:

Dependency	Purpose
------------	---------

Frontend framework/library	User interface
----------------------------	----------------

Backend framework Server-side processing

Database Data storage

Dependency Purpose

Database	Stores users, restaurants, menu items, orders, payments, and delivery records
----------	---

Authentication library	User authentication and access control
------------------------	--

Payment library	Payment processing and transaction handling
-----------------	---

Authentication library	User authentication
------------------------	---------------------

Testing framework Automated testing

The exact dependency list should be taken from the project's actual dependency file.

6.4 Usage Guide

The usage guide should explain:

Register

↓

Login

↓

Submit Order

↓

Select Restaurant and Menu



Track Order Status



Review Order Updates



Close / Confirm Resolution

6.5 Documentation Improvements The

documentation can be improved by:

- Adding screenshots.
- Adding architecture diagrams.
- Providing sample request inputs and outputs.
- Providing troubleshooting instructions.
- Listing all dependencies.
- Documenting API endpoints.
- Adding database schema information.
- Providing testing instructions.
- Adding contribution guidelines.

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

7.1 Overall Findings

The Smart Food Delivery and Order Management System addresses a practical food ordering problem by combining restaurant discovery, order placement, payment processing, delivery assignment, and order tracking.

The overall architecture should follow separation of concerns so that user-interface code, order processing, payment operations, database access, authentication, and notifications are independently maintained.

The major positive aspects are:

- Practical problem domain.
- Clear functional modules.
- Centralized order and delivery tracking.
- Potential for modular development.
- Git-based version control.
- Possibility of automated testing.
- Scope for future enhancement.

7.2 Code Quality Recommendations

The following improvements are recommended:

Recommendation 1 – Improve Modularity

Separate restaurant, order, payment, delivery, authentication, notification, and database operations into independent modules.

Recommendation 2 – Improve Validation

Implement both client-side and server-side validation for customer, cart, and order details.

Recommendation 3 – Improve Error Handling

Provide meaningful error messages instead of exposing technical errors to users.

Recommendation 4 – Add Automated Testing

Develop unit and integration tests for important ordering, payment, and delivery functions.

Recommendation 5 – Avoid Hard-Coded Request Rules

Request categories, priorities, and workflow statuses should be maintained in configuration files or database tables so that they can be updated easily.

Recommendation 6 – Improve Security

Sensitive configuration values such as passwords and database credentials should be stored using environment variables.

Recommendation 7 – Improve Documentation

Maintain a detailed README containing installation, configuration, usage, testing, and troubleshooting information.

Recommendation 8 – Improve Git Practices

Use feature branches and meaningful commit messages.

8. SOFTWARE ARCHITECTURE

8.1 Selected Architecture

A three-tier / layered architecture is suitable for the Smart Food Delivery and Order Management System.

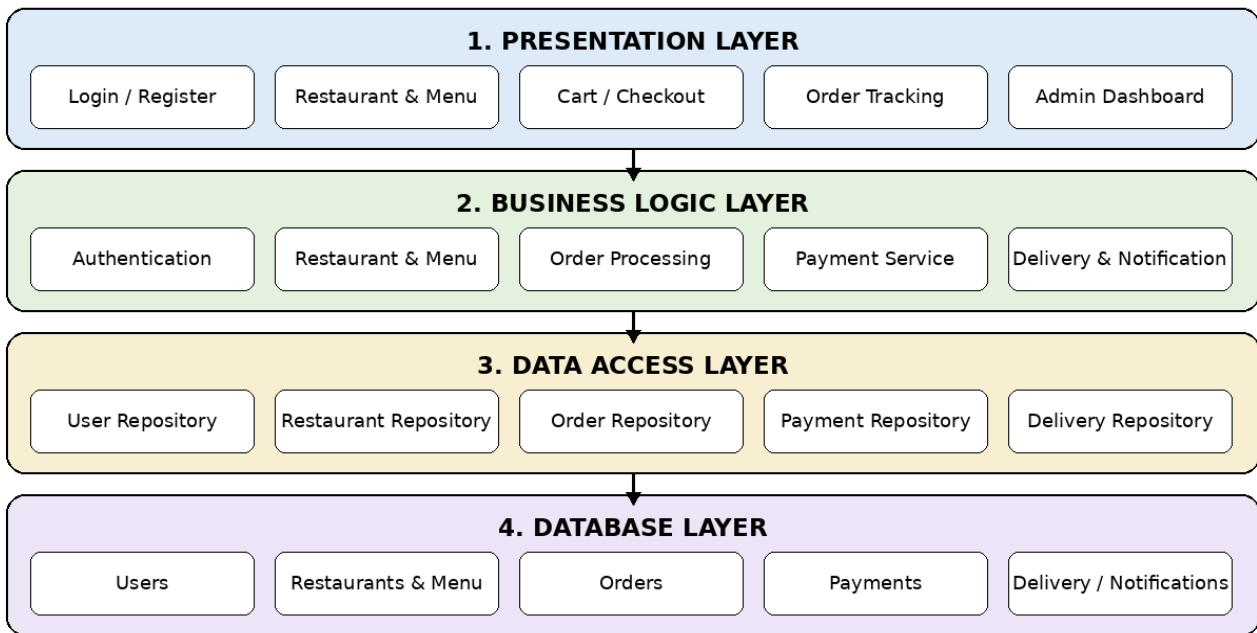
The three major layers are:

1. Presentation Layer.
2. Business Logic Layer.
3. Data Access Layer.

The architecture can be represented as:

ARCHITECTURE DIAGRAM

SMART FOOD DELIVERY AND ORDER MANAGEMENT SYSTEM



8.2 Presentation Layer

The presentation layer provides interaction between users and the application.

It contains:

- Login page.
- Registration page.
- Customer dashboard.
- Restaurant and menu browsing interface.
- Order tracking page.
- Delivery tracking interface.

8.3 Business Logic Layer This is the core of the application.

It performs:

- Order creation and tracking.
 - Restaurant, menu, and delivery-fee management.
 - Delivery partner assignment.

- Input validation.
- Authentication logic.
- Payment and notification processing.
- Report generation.

The business logic should be independent of the user interface wherever possible.

8.4 Data Access Layer

The data access layer manages communication between the application and database.

It handles:

- Creating records.
- Reading records.
- Updating records.
- Deleting records.
- Database queries.
- Data validation.

8.5 Database

The database stores information such as:

Users

Orders

Restaurants and Menu Categories

Delivery Assignments

Order Status History

Notifications

9. SYSTEM REQUIREMENT ANALYSIS

9.1 Functional Requirements

FR1 – User Registration

The system shall allow new users to create an account.

FR2 – User Login

The system shall authenticate registered users.

FR3 – Food Order Placement

The system shall allow users to submit delivery and orders.

FR4 – Restaurant and Menu Management

The system shall allow restaurant staff to manage menu items, prices, availability, and categories.

FR5 – Request Delivery Assignment

The system shall allow authorized staff to assign requests to delivery partners.

FR6 – Order Tracking

The system shall allow users to view the current status of their requests.

FR7 – Payment and Notifications

The system shall process supported payments and provide updates when order status changes.

FR8 – Data Management

The system shall allow authorized users to manage order records.

9.2 Non-Functional Requirements

Performance

The system should display request information within a reasonable response time.

Security

User accounts, payment information, and order data should be protected.

Reliability

The application should maintain consistent order status and order records.

Usability

The interface should be simple and understandable.

Maintainability

The application should use modular and well-documented code.

Scalability

The architecture should allow additional food deliverys and delivery modules to be added later.

10. COMPONENT INTERACTION

The major component interaction is:

USER

|

↓

+-----+

| Web / UI |

+-----+

|

↓

+-----+

| Authentication|

+-----+

|

↓

+-----+

| Request |

| Controller |

+-----+

/ | \

/ | \

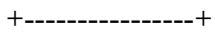
↓ ↓ ↓

Request Delivery Assignment Notification

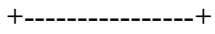
Service Service Service

\ | /

\ | /



| Database |



10.1 Order Interaction

User Input



Input Validation



Order Creation



Restaurant and Menu Selection



Delivery Assignment to Delivery Staff



Status Update



Notification



Order History

10.2 Delivery Planning Interaction

Orders

+

Restaurant Preparation Time and Delivery Address

+

Available Delivery Staff

↓

Delivery Assignment Planning

↓

Delivery Activity

↓

Resolution Update

↓

Order Summary

11. QUALITY ATTRIBUTE ANALYSIS

11.1 Security

The system should protect user and food delivery information using authentication, authorization, secure passwords, and protected configuration.

11.2 Performance

Order creation, payment confirmation, and dashboard queries should execute quickly.

Database queries should be optimized to avoid unnecessary operations.

11.3 Scalability

The system can be extended to support:

- Multiple restaurant partners.
- More restaurant categories.
- Preventive delivery modules.
- Asset tracking.
- Automated notifications.
- Service analytics.

11.4 Maintainability

Layered architecture makes it easier to modify one component without affecting unrelated components.

For example, changes to the request delivery assignment workflow should not require major changes to the user interface.

11.5 Usability

The application should provide:

- Simple forms.
- Clear labels.
- Meaningful error messages.
- Easy navigation.
- Clear order status indicators.

11.6 Reliability

Validation and automated testing can reduce incorrect records and application failures.

12. ARCHITECTURAL JUSTIFICATION

The layered architecture is selected because it provides a clear separation of responsibilities.

Reason 1 – Separation of Concerns

Each layer performs a specific responsibility.

Reason 2 – Maintainability

Changes can be made to individual modules without modifying the entire application.

Reason 3 – Testability

Business logic can be tested independently from the user interface.

Reason 4 – Scalability

New food delivery modules can be added without redesigning the complete system.

Reason 5 – Security

Authentication and authorization can be implemented as separate components.

Reason 6 – Reusability

Business services such as order processing, payment validation, and notifications can be reused by different interfaces.

13. FUTURE ENHANCEMENTS

The system can be enhanced with the following features:

1. AI-based restaurant and menu recommendations.
2. Dynamic delivery-time estimation.
3. Preventive delivery scheduling.

4. Delivery partner location tracking.
5. Predictive delivery alerts.
6. Graphical sales and order dashboards.
7. PDF order reports.
8. Email, SMS, and mobile notifications.
9. Mobile application.
10. Cloud deployment.
11. Multi-restaurant and multi-city management.
12. Advanced delivery analytics.

14. CONCLUSION

The Smart Food Delivery and Order Management System provides a practical software-based approach to reporting, tracking, and managing food ordering and delivery services.

The code review and repository evaluation demonstrate that software quality depends not only on application functionality but also on source-code organization, documentation, version control, testing, security, and maintainability.

A modular layered architecture provides a strong foundation for the application. Proper Git practices improve collaboration and traceability, while systematic testing helps verify request handling, status updates, and service workflows.

The recommended improvements include stronger automated testing, improved validation, better documentation, secure configuration management, modular business logic, meaningful Git commits, and flexible order-management rules.

Overall, the project provides a suitable foundation for a food delivery management application and can be extended with AI-based issue classification, predictive delivery, mobile services, analytics, and cloud-based deployment.

15. SCREENSHOT CHECKLIST

Add the following screenshots to make the report stronger:

1. GitHub repository home page
2. Repository folder structure
3. Commit history
4. Branches
5. README file
6. Application login page
7. Registration page
8. User dashboard
9. Restaurant and menu browsing page
10. Order tracking page
11. Request status update
12. Delivery assignment page
13. Service summary dashboard
14. Invalid input/error message
15. Database tables

Use captions such as:

Figure 1: GitHub Repository Structure

Figure 2: Git Commit History

Figure 3: User Login Interface

Figure 4: Restaurant and Menu Interface

Figure 5: Order Tracking Dashboard

Figure 6: Delivery Tracking Dashboard

16. FINAL EVALUATION MAPPING

Evaluation Criteria	Weight	Evidence to Include
Problem Analysis & Repository Organization	15%	Problem explanation + repository screenshot
Code Quality Review	25%	Code snippets + observations
Version Control Evaluation	20%	Git commits + branches screenshot
Testing & Validation	15%	Test cases + execution screenshots
Documentation & Repository Maintenance	15%	README + installation documentation
Findings & Recommendations	10%	Recommendations + conclusion
Total	100%	

Architecture Evaluation

Criteria	Marks
System Requirement Analysis	5
Selection of Software Architecture	5
Criteria	Marks
Architecture Diagram and Component Design	5
Component Interaction and Quality Attributes	5
Architectural Justification and Technical Presentation	5
TOTAL	25

REFERENCES

1. Git Documentation – Version control and repository management.
2. Software Engineering principles – modularity, maintainability, and software quality.
3. Project source code and Git repository.
4. Project README and technical documentation.
5. Database and framework documentation used in the implementation.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards,	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
	documentation, and maintainability with clear observations.			
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.

Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

